Privacy-Preserving Credit Card Fraud Detection and Visual Analytics using Python and Tableau

Milestone 2: Final Report, Demo and Presentation

(Chapters 3 to 5 and Appendix A)

CBCA293 Security and Privacy for Big Data Analytics, Bennett University

Rajat Sharma (S24BCAU0134), Ishan Chaudary (S24BCAU0002), Piyush Agarwal (S24BCAU0182), Suraj Tiwari (S24BCAU0136)

Faculty: Ms. Monia

Table of Contents

Chapter 3. Project implementation **1**

3.1 Development environment 1

3.2 Module description 2

3.3 Code implementation 3

3.4 System interface / screenshots 5

Chapter 4. Results and discussion **10**

4.1 Experimental setup 10

4.2 Output results 11

4.3 Performance analysis 14

4.4 Discussion 16

Chapter 5. Conclusion **18**

5.1 Conclusion 18

5.2 Limitations 19

5.3 Future scope 20

5.4 References 20

Appendix A. Additional screenshots **22**

A.1 k-anonymity before and after generalization 22

A.2 Threshold tuning on the training file 22

A.3 Project folder structure 23

A.4 Output files 23

A.5 Additional Tableau screenshots 24

Chapter 3. Project implementation

The Chapter 2 design is implemented in `src/`.

3.1 Development environment

Item	Version / detail	Role
Computer	MacBook Air, Apple Silicon (arm64), macOS	Development and running the pipeline
Python	3.13.15	Programming language
uv	installed on the Mac	Creates the virtual environment and installs the locked packages [1]
pandas	3.0.6	Data loading, grouping, percentiles, CSV output [2]
NumPy	2.5.3	Numeric support for pandas [3]
Matplotlib	3.11.2	Report charts [4]
hashlib, xml.etree.ElementTree, csv, json	Python standard library	SHA-256 hashing [5], writing the Tableau XML [6], reading/writing files
Tableau Desktop Public Edition	free edition, workbook saved locally	Opening and using the dashboard [7], [8]
Editor	VS Code and the macOS terminal	Writing and running code

This is a uv project. `pyproject.toml` lists the three libraries, `uv.lock` fixes their versions, and `.python-version` selects Python 3.13. Two commands set up and run the project:

```
uv sync                              # create .venv and install the locked packages
uv run python src/run_pipeline.py    # run steps 1 to 6
```

After the protected CSV exists, `uv run python src/build_dashboard.py` rebuilds only the Tableau workbook.

The code uses this folder structure:

```
CBCA-293-Security/
├── data/raw/            fraudTrain.csv, fraudTest.csv (Kaggle, never changed)
├── data/processed/      transactions_protected.csv (the only file Tableau reads)
├── src/                 the 8 Python modules
├── outputs/             data_profile.json, privacy_report.json, rule_tuning.csv,
│                        metrics.json, charts/*.png
├── dashboard/           fraud_dashboard.twb, screenshots/
└── reports/             this report
```

3.2 Module description

Each pipeline step has its own module. Table 3.1 lists its inputs, outputs and main functions.

Table 3.1: Modules of the system

Module	Step	Input	Output	Main functions
`config.py`	Settings	none	Settings used by all modules	`SALT`, `K_ANONYMITY`, `AGE_BINS`, `NARROW_AGE_BINS`, `STATE_TO_REGION`, rule settings
`load_data.py`	1. Load	`data/raw/*.csv`	Combined table, `outputs/data_profile.json`	`load_raw()`, `profile()`
`privacy.py`	2. Privacy layer	Combined raw table	Protected table, `outputs/privacy_report.json`	`mask_card()`, `pseudonymize()`, `age_at_transaction()`, `k_anonymity()`, `protect()`
`fraud_rules.py`	3. Rules, 4. Evaluate	Protected table	Rule columns, `outputs/rule_tuning.csv`, `outputs/metrics.json`	`learn_limits()`, `apply_rules()`, `score()`, `tune_threshold()`, `evaluate()`, `add_decision()`
`make_charts.py`	5. Charts	Protected table, limits, metrics, privacy report	5 PNG files in `outputs/charts/`	`fraud_by_hour()`, `fraud_by_category()`, `rule_comparison()`, `confusion_matrix()`, `k_anonymity()`
`build_dashboard.py`	6. Tableau workbook	`transactions_protected.csv`	`dashboard/fraud_dashboard.twb`	`build_datasource()`, `build_shared_views()`, `build_worksheet()`, `build_dashboard()`, `build()`, `validate()`
`make_extract.py`	6. Tableau extract	`transactions_protected.csv`	`transactions_protected.hyper`	`make_extract()`
`run_pipeline.py`	All	Raw files	All outputs	`main()`, `save_json()`

`config.py` holds the salt, k = 5, age bands, state-to-region map and rule factors. The factors are the 99th percentile, 2 times the overall fraud rate and 5 times the usual spend. Other modules import these values.

3.3 Code implementation

The main operations of each module are described below; the full commented source code is in `src/`.

3.3.1 Loading and cleaning (`load_data.py`)

`load_raw()` reads both files and stacks them into one table. `dataset_split` records the source file for each row. `usecols` skips the unnamed row-number field, while pandas stores text fields with few distinct values as `category` to reduce memory use. It also removes the simulator's `fraud_` prefix from merchant names.

3.3.2 Masking (`privacy.py`)

The masked value keeps only the last four digits, within the PCI DSS display rule [9]. Its format stays the same for any input length.

3.3.3 Pseudonymization with salted SHA-256 (`privacy.py`)

The function places the salt before the card number and hashes the combined text with SHA-256 [5], [10]. The first 10 hexadecimal characters form the `customer_id`, such as `C_3298fc2482`. `protect()` hashes each of the 999 distinct cards once, then maps the result to its rows. It does not repeat the hash across all 1.85 million rows.

3.3.4 Generalization (`privacy.py` and `config.py`)

Age is calculated at the transaction date by dividing the days since birth by 365.25 and keeping whole years. `pd.cut` assigns one of 5 bands. The state code becomes its US Census region [11]. The function builds the protected table one column at a time. Names, address, coordinates, job and other omitted fields never enter it.

3.3.5 k-anonymity check (`privacy.py`)

The function groups the data by its quasi-identifiers and counts distinct customers in each group [12]. Counting rows would overstate group size because each customer has many transactions. It lists groups below k and counts their customers as "exposed." `protect()` runs the check first on narrow age band, gender and state, then on age band, gender and region.

3.3.6 Learning the rule limits (`fraud_rules.py`)

This function receives training rows only. It calculates each category's 99th amount percentile, finds hours and categories with at least 2 times the overall fraud rate, and records the median spend for every customer and category. The night hours come from that rate test rather than a fixed list.

3.3.7 Applying the rules (`fraud_rules.py`)

Each rule returns 0 or 1. `fillna` assigns the category median as the usual spend when a customer has no training history. Adding the four columns gives the risk score.

3.3.8 Scoring and threshold tuning (`fraud_rules.py`)

`score()` builds the confusion matrix and metrics from Section 2.4.7, including guards against division by zero. `tune_threshold()` tests thresholds 1 through 4 on the training rows and keeps the highest F1. On the test rows, `evaluate()` scores each rule, the combined decision and a "flag nothing" baseline. The baseline uses `risk_score > 99`, which is always false. `add_decision()` then writes `flagged` and one of four `outcome` labels: `Caught`, `Missed`, `False alarm` or `Correctly passed`.

3.3.9 Keeping raw data out of later steps (`run_pipeline.py`)

After `privacy.protect()` returns, the pipeline drops its reference to the raw table. Steps 3 through 6 receive only the protected table, matching the trust boundary in Figure 2.1. The protected CSV has exactly 19 columns (Section 4.2).

3.3.10 Generating and validating the Tableau workbook (`build_dashboard.py`)

A Tableau workbook (`.twb`) is an XML text file. `build_dashboard.py` writes it with Python's `xml.etree.ElementTree` module [6]. One command can rebuild the workbook. The script defines:

- a text data-source connection to the 19 columns in `transactions_protected.csv`;
- six KPI calculated fields in Tableau's formula language, for example Rule Recall % = SUM([flagged] * [is_fraud]) / SUM([is_fraud]) and Rule Precision % = SUM([flagged] * [is_fraud]) / SUM([flagged]);
- six shared filters for date range, category, region, age band, gender and dataset split, applied to every worksheet that uses the source;
- twelve worksheets: 6 KPI tiles (Total Transactions, Fraud Transactions, Fraud Rate %, Fraud Amount $, Rule Recall %, Rule Precision %) and 6 charts (Fraud Rate by Hour, Fraud Count by Category, Fraud by Age Band, Fraud by Region, Monthly Fraud Trend,

Rule Outcomes);

- a 1265 x 760 pixel dashboard (it fits a laptop screen in Presentation Mode) named "Fraud & Privacy Dashboard," with a title and team names, 6 KPI tiles, a 2 x 3 chart grid and a filter panel.

The "Rule Outcomes" chart omits the much larger `Correctly passed` group so the `Caught`, `Missed` and `False alarm` bars remain readable. "Fraud by Age Band" uses a manual order because alphabetical sorting would put `<25` last.

After writing the file, `build()` calls `validate()` and reads the XML back. It checks that the XML parses, referenced fields exist, worksheets declare their fields, the dashboard contains every worksheet, layout zones stay inside their parents and each sheet has a tab. A failed check stops the script. The final run printed: `Validated fraud_dashboard.twb: 12 worksheets, 6 calculated fields, 6 shared filters.`

Tableau Public only opens workbooks whose data is an extract, so `make_extract.py` copies the protected CSV into a Tableau extract (`transactions_protected.hyper`, 1,852,394 rows) with the Tableau Hyper API. `build_dashboard.py` links the workbook to it and also writes `fraud_dashboard.twbx`, one packaged file holding the dashboard and the extract. The `.twb` stores an absolute path to the data folder. On another computer, `--tableau-data-dir` rebuilds it with the correct folder.

3.4 System interface / screenshots

3.4.1 Pipeline console output

The terminal command is `uv run python src/run_pipeline.py`. Its final output is below.

```
[1/6] Loading raw Kaggle files ...
   1,852,394 rows, 9,651 fraud (0.52%)
   saved outputs/data_profile.json
[2/6] Privacy layer: mask, hash, generalize, suppress ...
   k-anonymity before: FAIL (smallest group 1, 659 customers in groups < 5)
   k-anonymity after: PASS (smallest group 5, 0 customers in groups < 5)
   saved outputs/privacy_report.json
[3/6] Learning rule limits from the TRAIN file ...
   night hours [0, 1, 2, 3, 22, 23], risky categories ['grocery_pos', 'misc_net',
'shopping_net']
   best threshold on train: risk_score >= 3
[4/6] Evaluating on the unseen TEST file ...
   precision 34.1%, recall 57.1%, F1 0.427
   saved outputs/metrics.json
[5/6] Drawing report charts ...
   saved outputs/charts/*.png
[6/6] Writing protected CSV and Tableau workbook ...
   saved data/processed/transactions_protected.csv
```

```
Validated fraud_dashboard.twb: 12 worksheets, 6 calculated fields, 6 shared filters.
   saved dashboard/fraud_dashboard.twb
Done in 114 seconds.
```

Figure 3.1: Console output of the complete pipeline run

The MacBook Air processed 1.85 million raw rows and wrote the Tableau workbook in 114 seconds. The current version of step 6 also writes the Tableau extract and the packaged `fraud_dashboard.twbx` (Section 3.3.10), so its last lines name those two files as well.

3.4.2 Report charts produced by the pipeline

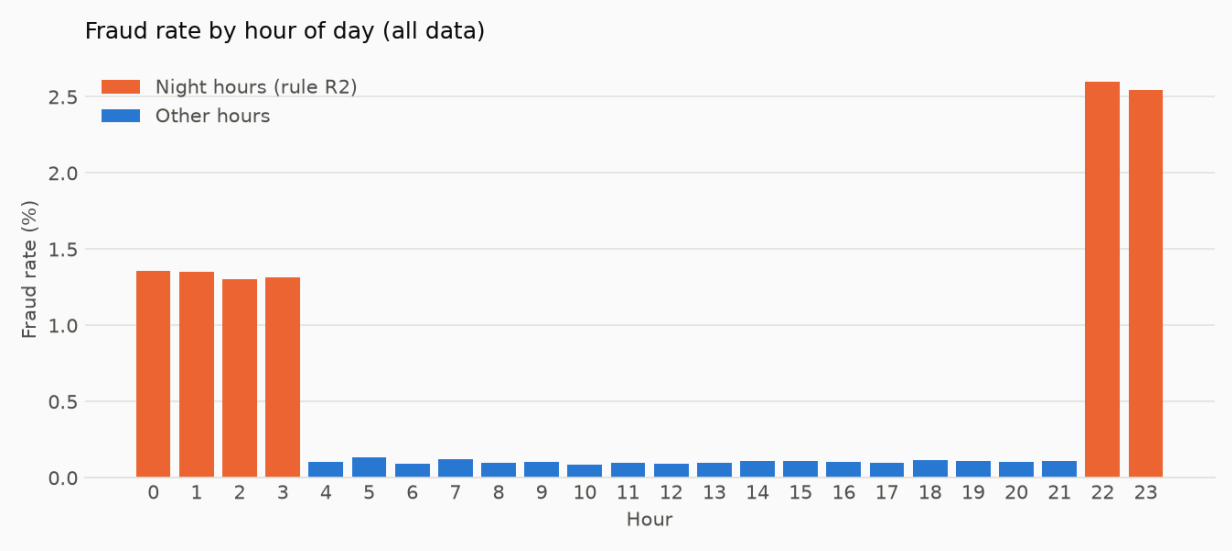


Figure 3.2: Fraud rate by hour of day (all data). Orange bars are the night hours learned for rule R2.

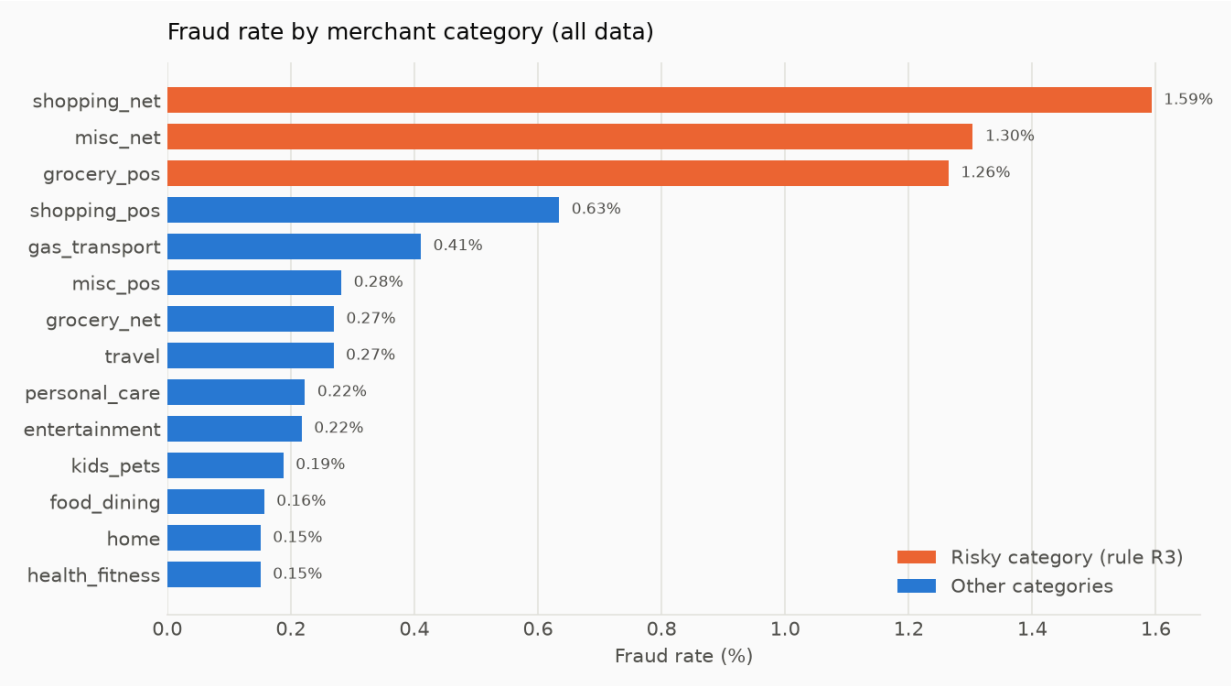


Figure 3.3: Fraud rate by merchant category (all data). Orange bars are the risky categories learned for rule R3.

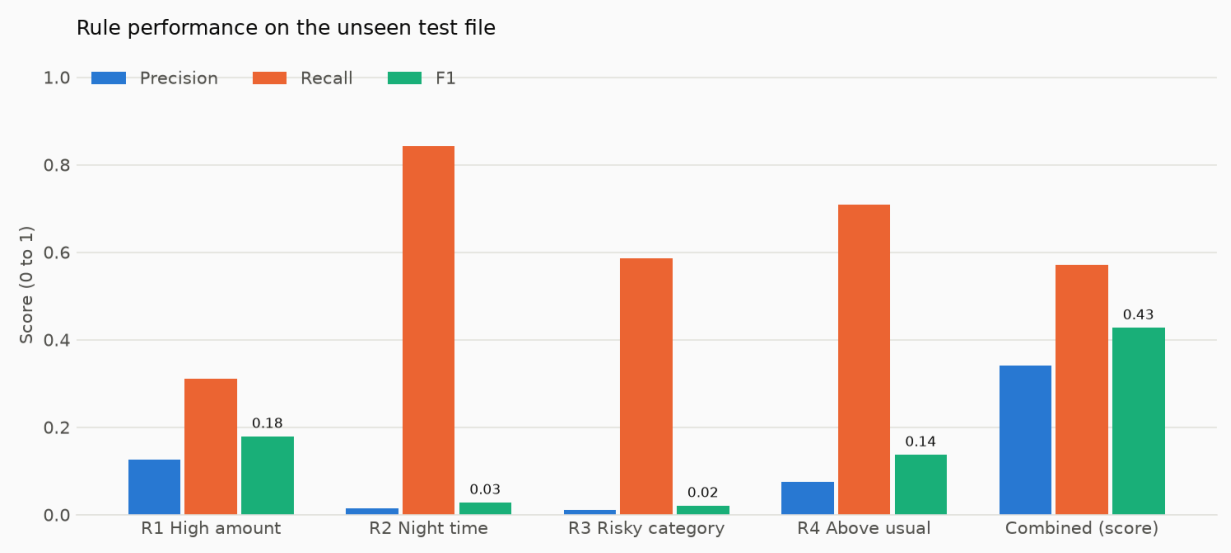


Figure 3.4: Precision, recall and F1 of each rule and of the combined score on the test file

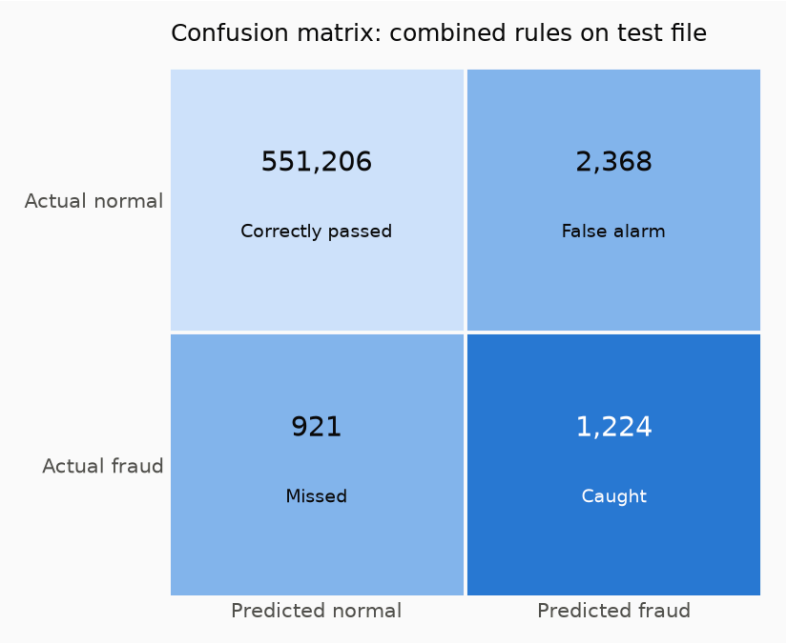


Figure 3.5: Confusion matrix of the combined rules on the test file

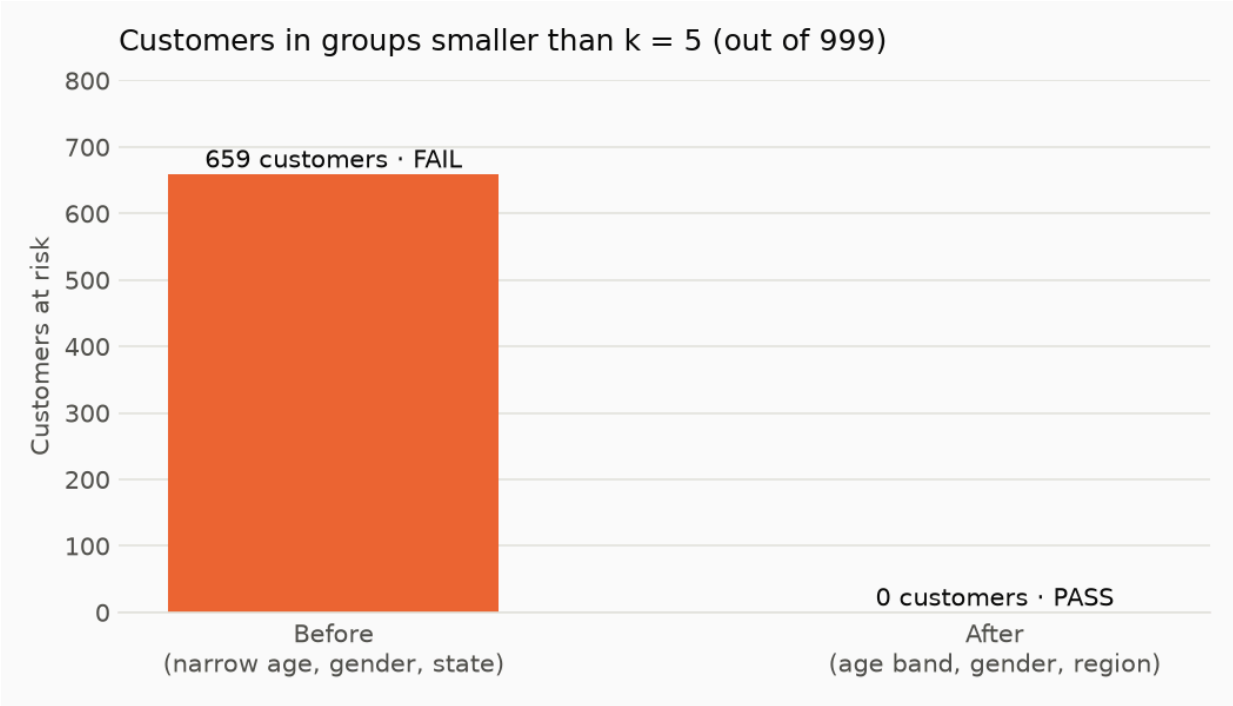


Figure 3.6: Customers in groups smaller than k = 5, before and after generalization

Chapter 4 discusses these results.

3.4.3 Tableau dashboard

`dashboard/fraud_dashboard.twbx` opens in Tableau Desktop Public Edition; Figures 3.7 and 3.8 were captured from it in Presentation Mode. Figure 3.7 uses all rows. Figure 3.8 sets Dataset Split to `test`, which makes the KPIs match Chapter 4.

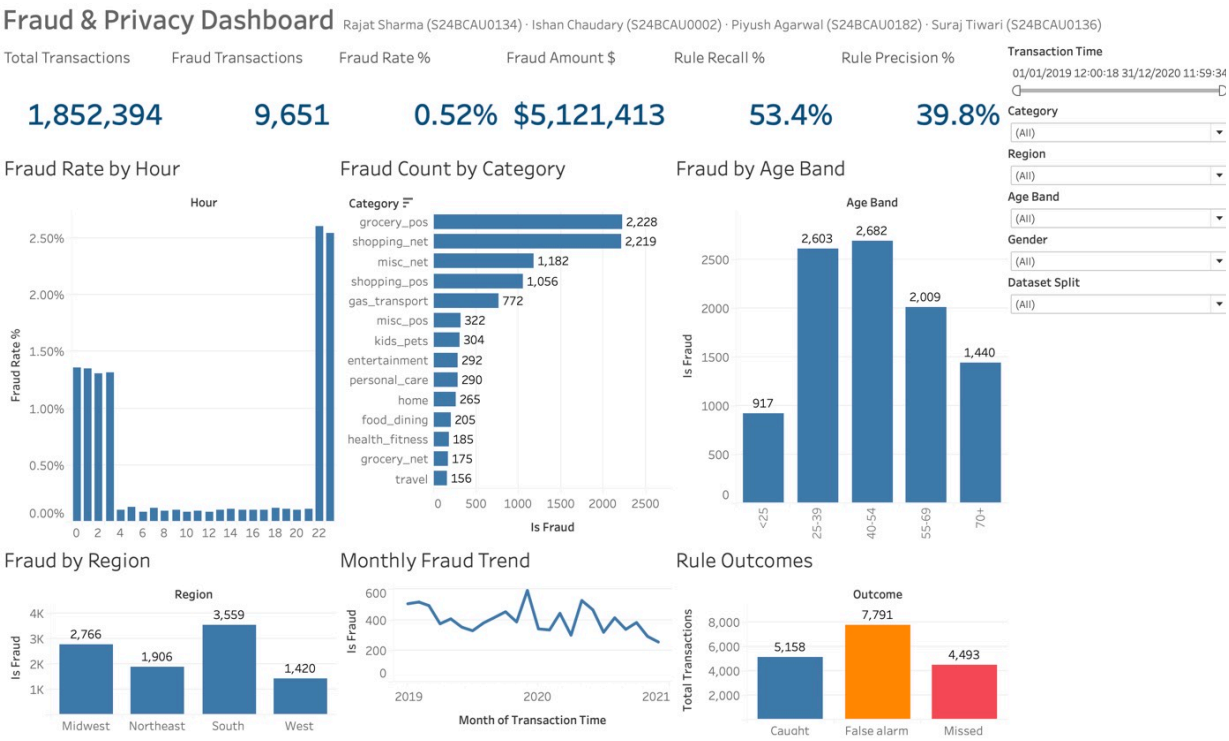


Figure 3.7: Fraud & Privacy Dashboard, all data (train and test)

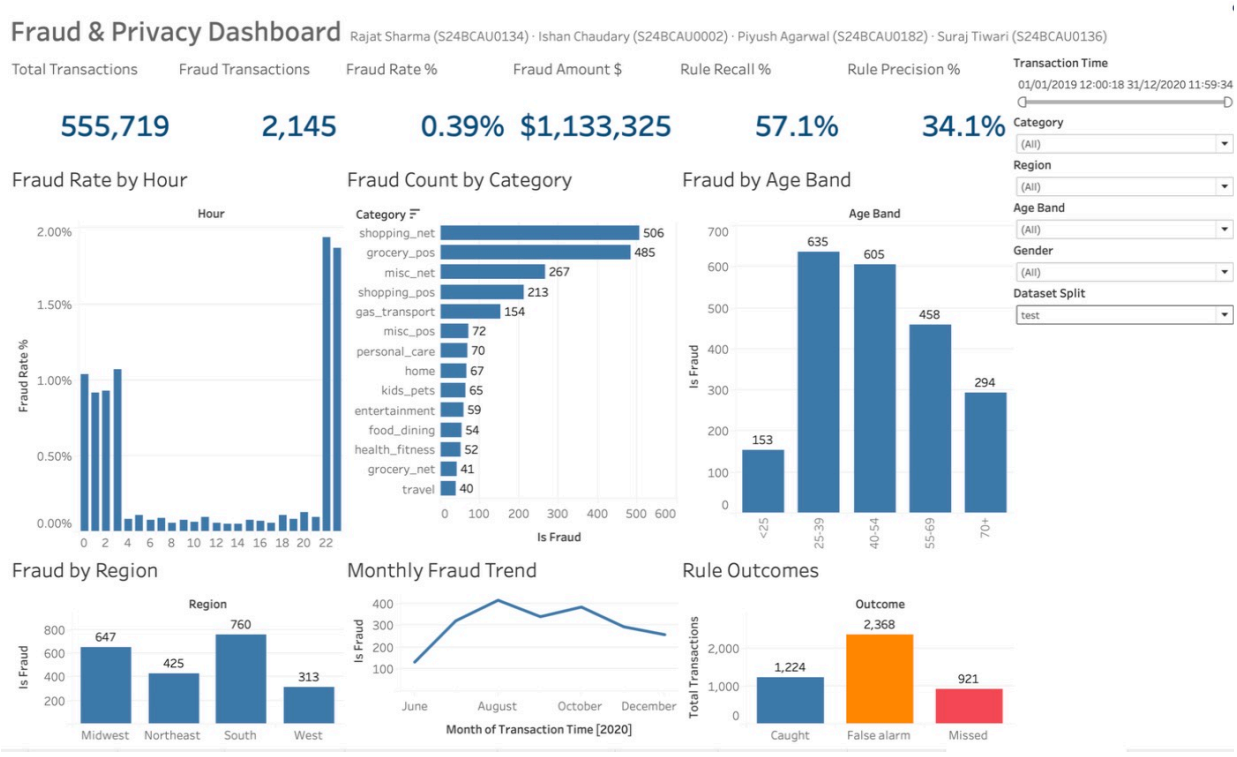


Figure 3.8: Fraud & Privacy Dashboard with Dataset Split = test

Six KPI tiles run across the top. Six charts fill the middle and bottom rows, and the right panel contains 6 filters that update every view. Only protected fields appear. A customer is represented by a pseudonym and masked card number, a location by its region, and an age by its band.

Chapter 4. Results and discussion

The results come from `outputs/`, the console output in Figure 3.1 and, where stated, row counts from the protected CSV.

4.1 Experimental setup

The run uses the protocol from Section 2.6.

Item	Value
Hardware	MacBook Air, Apple Silicon (arm64), macOS
Software	Python 3.13.15, pandas 3.0.6, NumPy 2.5.3, Matplotlib 3.11.2, run with uv
Training data	`fraudTrain.csv`: 1,296,675 rows, 7,506 fraud (0.58%), 2019-01-01 00:00:18 to 2020-06-21 12:13:37
Test data	`fraudTest.csv`: 555,719 rows, 2,145 fraud (0.39%), 2020-06-21 12:14:25 to 2020-12-31 23:59:34
Split	By time, as given by the two files. No shuffling.
Learned from train only	Overall fraud rate, category 99th percentiles, night hours, risky categories, customer medians, category medians, score threshold
Reported on	Test rows only
Run time	114 seconds for all six steps

The test file starts 48 seconds after the training file ends. Every test transaction is later than the data used to learn the rules.

What was learned from the training file

`metrics.json` gives a training fraud rate of 0.5789%. R2 and R3 mark an hour or category as risky at twice that rate, or 1.158% and above.

Table 4.1: Rule limits learned from the training file

Rule	Learned value	Evidence
R1 High amount	One 99th percentile amount per category (14 limits)	Kept in memory by `learn_limits()`
R2 Night time	Hours 0, 1, 2, 3, 22 and 23	`metrics.json`, console
R3 Risky category	`grocery_pos`, `misc_net`, `shopping_net`	`metrics.json`, console

Rule	Learned value	Evidence
R4 Above usual spend	Median spend of each of the 983 customers in the training file; category median for others	Counted from the protected CSV

The six selected night hours have training fraud rates from 1.42% to 2.88%, with hour 22 highest. Every other hour is between 0.10% and 0.14%. The selected categories are `shopping_net` at 1.76%, `misc_net` at 1.45% and `grocery_pos` at 1.41%. The next category, `shopping_pos`, is below the 1.158% cutoff at 0.72%. Of the 924 customers in the test file, 16 are absent from training. R4 uses the category median for them.

Threshold tuning on the training file

`outputs/rule_tuning.csv` compares thresholds 1 through 4 on the training rows.

Table 4.2: Threshold tuning on the training file (1,296,675 rows, 7,506 fraud)

Threshold (flag if risk_score ≥)	Flagged	TP	FP	FN	Precision	Recall	F1	Accuracy
1	530,962	7,208	523,754	298	1.36%	96.03%	0.0268	59.58%
2	108,984	5,411	103,573	2,095	4.96%	72.09%	0.0929	91.85%
3	**9,357**	**3,934**	**5,423**	**3,572**	**42.04%**	**52.41%**	**0.4666**	**99.31%**
4	1,485	1,237	248	6,269	83.30%	16.48%	0.2752	99.50%

Threshold 3 has the highest F1 at 0.4666, so the pipeline uses **risk_score ≥ 3**. At threshold 1, recall reaches 96.03% but produces 523,754 false alarms. Threshold 4 raises precision to 83.30% but catches only 16.48% of the fraud.

4.2 Output results

4.2.1 Dataset profile

`outputs/data_profile.json`, written by `load_data.py`, contains the following counts.

Fact	Value
Total rows	1,852,394
Fraud rows	9,651 (0.52%)
Missing values	0
Duplicate rows (full row)	0
Duplicate transaction IDs	0
Customers (cards)	999
States / categories / merchants	51 / 14 / 693

Fact	Value
Customer age at transaction	13 to 96

4.2.2 Privacy results: one record before and after

Table 4.3 takes the first record from `outputs/privacy_report.json` before and after protection. The simulated row does not describe a real person.

Table 4.3: Example record before and after the privacy layer

Field	Before (raw)	After (protected)	Technique
Card number	`2703186189652095`	`**** **** **** 2095`	Masking
Customer identifier	(the card number)	`C_3298fc2482`	Salted SHA-256 pseudonym
Name	Jennifer Banks	removed	Suppression
Date of birth	1988-03-09	`25-39`	Generalization to age band
Street, city, ZIP	561 Perry Cove, Moravian Falls, 28654	removed	Suppression
State	NC	South	Generalization to Census region

Sixteen raw columns do not reach the output unchanged. The pipeline deletes 13 of them: `first`, `last`, `street`, `city`, `zip`, `lat`, `long`, `city_pop`, `job`, `merch_lat`, `merch_long`, `trans_num` and `unix_time`. It replaces `cc_num` with `card_masked` and `customer_id`, `dob` with `age_band`, and `state` with `region`.

4.2.3 k-anonymity before and after generalization

Table 4.4: k-anonymity check with k = 5 (999 customers)

Measure	Before	After
Quasi-identifiers	narrow age band (7 bands), gender, state	age band (5 bands), gender, region
Number of groups	461	40
Smallest group (customers)	1	5
Groups smaller than 5	395	0
Customers in groups smaller than 5	659 (66% of 999)	0
Result	**FAIL**	**PASS**

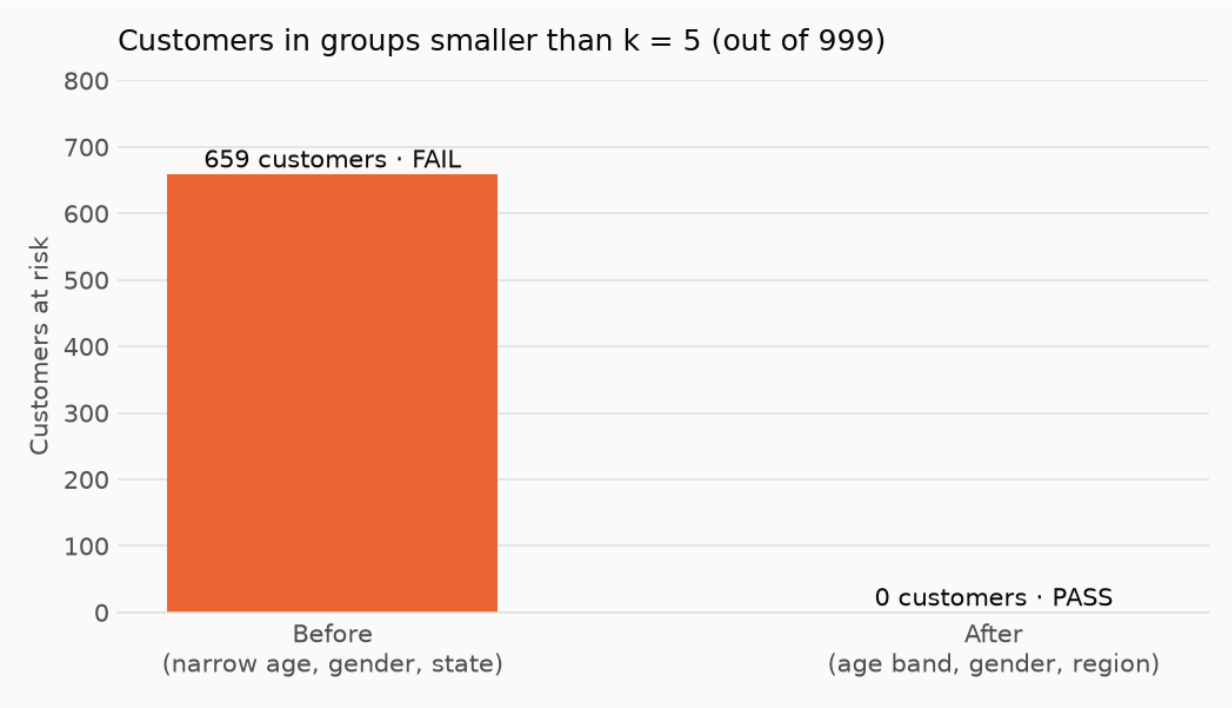


Figure 4.1: Customers in groups smaller than k = 5, before and after generalization

Exact states and narrow age bands split the data into 461 groups. Of these, 395 have fewer than 5 customers, and the smallest has one. That customer can be singled out by age band, gender and state. Generalizing to 4 regions and 5 age bands produces all 40 possible combinations (5 x 2 x 4). The smallest contains 5 customers, and none falls below k. This meets Objective 4.

4.2.4 Files produced

Output	Content
`data/processed/transactions_protected.csv`	1,852,394 rows, 19 columns, about 265 MB. The only input of Tableau.
`outputs/data_profile.json`	Dataset facts (Section 4.2.1)
`outputs/privacy_report.json`	k-anonymity before and after, example record, removed columns
`outputs/rule_tuning.csv`	Table 4.2
`outputs/metrics.json`	Learned rule settings and all test metrics (Section 4.3)
`outputs/charts/*.png`	Figures 3.2 to 3.6
`data/processed/transactions_protected.hyper`	Tableau extract of the protected CSV (Tableau Public needs an extract)
`dashboard/fraud_dashboard.twbx`	Packaged Tableau workbook (dashboard + extract): 12 worksheets, 6 calculated fields, 6 shared filters, validated

The protected CSV has the 19 columns listed in `docs/01_project_understanding.md`, Section 4: `transaction_time`, `hour`, `dataset_split`, `customer_id`, `card_masked`, `merchant`, `category`, `amount`, `gender`, `age_band`, `region`, the four rule columns, `risk_score`, `flagged`, `is_fraud` and `outcome`.

4.3 Performance analysis

4.3.1 Test results for each rule and the combined score

Table 4.5: Results on the unseen test file (555,719 rows, 2,145 fraud)

Rule	Flagged	TP (caught)	FP (false alarms)	FN (missed)	Precision	Recall	F1	Accuracy
R1 High amount	5,315	666	4,649	1,479	12.53%	31.05%	0.1786	98.90%
R2 Night time	130,139	1,807	128,332	338	1.39%	84.24%	0.0273	76.85%
R3 Risky category	121,699	1,258	120,441	887	1.03%	58.65%	0.0203	78.17%
R4 Above usual spend	20,073	1,518	18,555	627	7.56%	70.77%	0.1366	96.55%
Combined (risk_score ≥ 3)	**3,592**	**1,224**	**2,368**	**921**	**34.08%**	**57.06%**	**0.4267**	**99.41%**
Flag-nothing baseline	0	0	0	2,145	0.00%	0.00%	0.0000	99.61%

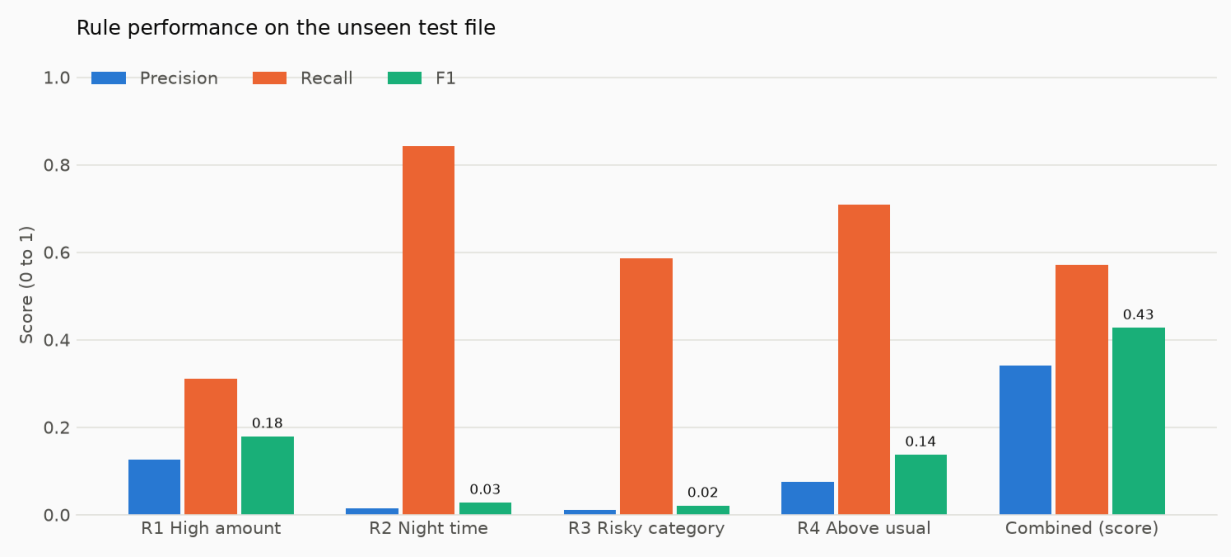


Figure 4.2: Precision, recall and F1 on the test file

4.3.2 Confusion matrix of the combined rules

	Actually fraud	Actually normal
Flagged	1,224 (Caught)	2,368 (False alarm)
Not flagged	921 (Missed)	551,206 (Correctly passed)

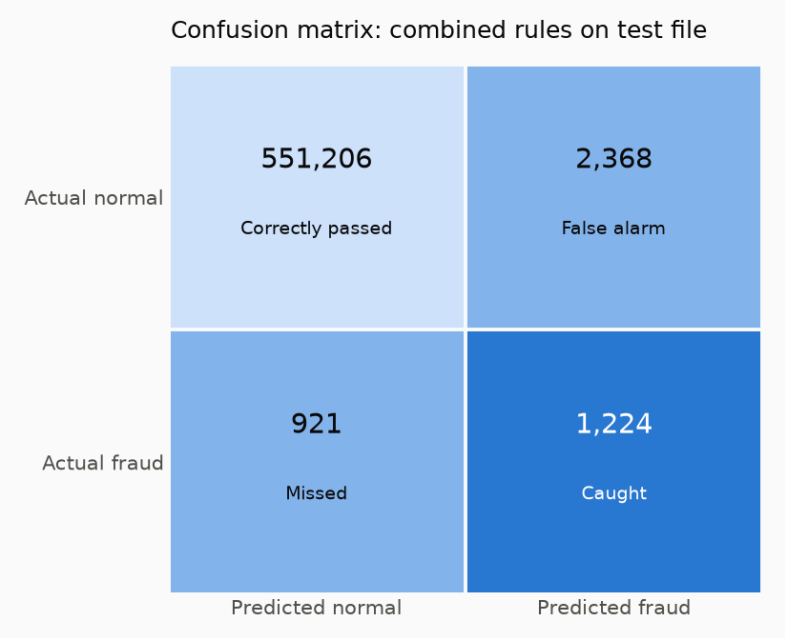


Figure 4.3: Confusion matrix of the combined rules on the test file

The combined rules flag 3,592 transactions, or 0.65% of the test file. They catch 1,224 frauds, which is 57.1% of all test fraud. One in 3 alerts is correct (34.1%). The test fraud rate is 0.39%, so a flagged transaction is about 88 times more likely to be fraud than a random test transaction.

4.3.3 Why accuracy is misleading

The flag-nothing baseline has **99.61% accuracy**, above the combined rules at 99.41%, but its recall is **0%**. It catches no fraud. Since 99.6% of rows are normal, accuracy mainly measures how often normal rows are left alone [13]. R2 and R3 show the opposite failure. Their recall is 84.2% and 58.7%, but they flag about 23% and 22% of all test transactions, leaving precision close to the base rate. Precision, recall and F1 give the useful comparison.

4.3.4 Why combining the rules raises precision

No single rule is precise. R2 and R3 each fire on more than 120,000 test transactions, almost all normal. R1 and R4 are narrower but remain below 13% precision. A normal transaction may trigger one rule, such as a late-night online purchase, but few trigger three. Only 2,368 of 553,574 normal test transactions, or 0.43%, reach a score of 3. By comparison, 1,224 of 2,145 frauds, or 57.1%, reach that score. Requiring 3 rules removes most normal transactions while retaining more than half the fraud.

The combined score has 2.7 times the precision of the best single rule, 34.1% against 12.5% for R1. Its F1 is 2.4 times as high, 0.427 against 0.179.

4.3.5 Train compared with test

Data	Precision	Recall	F1
Train (threshold 3)	42.04%	52.41%	0.4666
Test (threshold 3)	34.08%	57.06%	0.4267

Test F1 falls from 0.4666 to 0.4267, while recall rises from 52.41% to 57.06%. The rules still detect later transactions rather than only fitting the training rows. Precision falls partly because fraud is less common in the test file, 0.39% against 0.58%, so each alert has fewer frauds available to find.

4.4 Discussion

The 114-second run removed every direct identifier and measured k-anonymity at k = 5. Four one-line rules reached an F1 of 0.427 on later test data. An alert can be traced to its conditions, such as a large amount at night in an online shopping category. The generated Tableau workbook passed its structural checks and can be rebuilt from the protected CSV.

The threshold controls the balance between missed fraud and investigator workload. At 3, the test run produces 2,368 false alarms and misses 921 frauds, about 2 false alarms per fraud caught. On training data, threshold 2 reaches 72% recall but creates more than 10 times as many alerts. Threshold 4 reaches 83% precision and misses most fraud. F1 selects threshold 3 here. A bank would instead compare the cost of missed fraud with the cost of checking an alert [14], [15].

Generalization removes state-level and exact-age analysis from the dashboard. It keeps only 4 regions and 5 age bands. The detailed version left 659 customers in groups below 5, so that loss of detail was needed to meet the chosen k. The fraud rules use amount, hour, category and customer pseudonym, none of which the generalization removes. Their inputs are unchanged by this privacy step.

A tested distance rule did not separate the classes. The average is 76.1 km for normal transactions and 76.3 km for fraud. Dropping that rule also allowed the pipeline to remove the home and merchant coordinates.

By default, the dashboard combines training and test rows. The Rule Recall % tile then shows 53.4%, or 5,158 of 9,651 frauds caught. Rule Precision % shows 39.8%, or 5,158 of 12,949 alerts. Tables 4.2 and 4.5 provide those counts. They are not test results because the rules learned from the training rows. Setting **Dataset Split** to `test` changes the tiles to 57.1% recall and 34.1% precision, matching Table 4.5 and Figure 3.8.

These numbers describe simulated data, not expected performance at a bank. Real fraud is more varied and changes over time [14]. Section 5.2 covers the remaining limits.

Chapter 5. Conclusion

5.1 Conclusion

The Python pipeline processed 1,852,394 simulated card transactions. It masked and pseudonymized card numbers, generalized dates of birth and states, and removed the remaining personal fields. The released data passed a k-anonymity check with k = 5. Four readable rules scored suspicious transactions, and the generated Tableau workbook reads only the protected file.

The detailed privacy design did not pass. With exact states and narrow age bands, 659 of 999 customers were in groups smaller than 5. After generalization to 4 regions and 5 age bands, every group contained at least 5 customers.

The combined rules were learned from the training file only. On the later test file they caught 57.1% of fraud at 34.1% precision, with F1 0.427. The best single rule reached F1 0.179.

Accuracy hid the failure of the baseline. Flagging nothing scored 99.61% accuracy and caught no fraud.

None of the four rules uses names, addresses, exact ages or states. Generalizing or removing those fields did not change the rule inputs.

Table 5.1 checks each objective from Section 1.3 against the measured result.

Table 5.1: Objectives and results

No.	Objective (Section 1.3)	Measured result	Status
1	Load and profile both files (1,852,394 rows), confirm missing values and duplicates, save `data_profile.json`	1,852,394 rows loaded; 0 missing values; 0 duplicate rows; 0 duplicate transaction IDs; profile saved	Met
2	No direct identifiers in the file given to Tableau	Protected CSV has 19 columns with no name, address, coordinates, job, date of birth or full card number; 13 columns suppressed, 3 transformed	Met
3	Show at most the last 4 digits; replace the card number with a salted SHA-256 pseudonym	`**** **** **** 2095` and `C_3298fc2482` for the first record; 999 cards each mapped to one pseudonym	Met
4	k-anonymity with k ≥ 5 on (age band, gender, region), before and after checks saved	Before: FAIL, smallest group 1, 659 customers exposed. After: PASS, smallest group 5, 0 customers exposed	Met
5	Four rules learned from train only; threshold with best train F1 saved	4 rules implemented; best threshold risk_score ≥ 3 (train F1 0.4666); `rule_tuning.csv` saved	Met

No.	Objective (Section 1.3)	Measured result	Status
6	Evaluate on the 555,719 unseen test rows with precision, recall, F1, accuracy, confusion matrix, and a flag-nothing baseline	Combined: precision 34.08%, recall 57.06%, F1 0.4267, accuracy 99.41%; TP 1,224, FP 2,368, FN 921, TN 551,206; baseline accuracy 99.61% with 0% recall	Met
7	Tableau dashboard on the protected file with ≥ 6 KPIs, ≥ 6 charts, ≥ 5 filters	6 KPIs, 6 charts, 6 shared filters; workbook passed `validate()` (12 worksheets, 6 calculated fields) and was opened in Tableau Public (Figures 3.7 and 3.8)	Met
8	Whole pipeline runs from one command and repeats its outputs	`uv run python src/run_pipeline.py` runs all 6 steps in about 2 minutes; no step uses random numbers and the salt is fixed	Met

All eight objectives are met and measured.

5.2 Limitations

1. Sparkov generated the dataset [16], [17]. Real fraud is more varied and changes over time [14], so the measured precision and recall do not predict performance at a bank.
2. The four rules catch patterns like the ones learned. They miss 921 test frauds that do not combine three warning signs. Machine learning models may find more complex patterns [15], [18], though their decisions are harder to explain.
3. The test run raises about 2 false alarms for every fraud caught, 2,368 against 1,224. Each alert would use investigator time in a bank [15].
4. k-anonymity guarantees group size, not protection from attribute disclosure. If all 5 customers in a group share a sensitive value, an attacker learns it. Outside knowledge may also narrow the group [19]. The pipeline does not test l-diversity.
5. The selected quasi-identifiers are age band, gender and region. Merchant, exact transaction time and amount remain in the file. An attacker who knows some purchases could use those fields too.
6. `src/config.py` contains the salt so repeated runs produce the same output. Anyone with the code and card numbers can rebuild the pseudonyms. A real deployment would store the salt in a secrets vault and keep it apart from the data [20].
7. A workbook published to a Tableau Public profile is available to anyone [8]. This workbook must use only the protected CSV and remain local or move through a private channel.
8. The pipeline processes files rather than a live payment stream, and it learns the rule limits once. A deployed system would need to update them as fraud patterns change [14].

9. The `.twb` file stores the CSV folder as an absolute path. Another computer must rebuild it with `--tableau-data-dir`.

5.3 Future scope

1. Check l-diversity [19] for the fraud label within each group, and test differential privacy [21] for aggregate dashboard KPIs.
2. Train a decision tree or random forest on the protected features and compare it with the rules. Industry systems often combine rules and models [15].
3. Choose the threshold from the estimated money saved and the cost of reviewing alerts, not F1 alone [14].
4. Read the salt from an environment variable or secrets manager instead of source code.
5. Test frequency rules, such as several payments from one card within a short period.
6. Apply the rules to a transaction stream with a tool such as Apache Spark.
7. Add unit tests for functions including `mask_card()`, `pseudonymize()` and `k_anonymity()`.

5.4 References

[1] Astral, "uv: An extremely fast Python package and project manager," Documentation. [Online]. Available: https://docs.astral.sh/uv/ (accessed Oct. 8, 2026).

[2] pandas development team, "pandas documentation." [Online]. Available: https://pandas.pydata.org/docs/ (accessed Oct. 8, 2026).

[3] NumPy developers, "NumPy documentation." [Online]. Available: https://numpy.org/doc/stable/ (accessed Oct. 8, 2026).

[4] Matplotlib development team, "Matplotlib documentation." [Online]. Available: https://matplotlib.org/stable/ (accessed Oct. 8, 2026).

[5] Python Software Foundation, "hashlib: Secure hashes and message digests," Python Documentation. [Online]. Available: https://docs.python.org/3/library/hashlib.html (accessed Oct. 8, 2026).

[6] Python Software Foundation, "xml.etree.ElementTree: The ElementTree XML API," Python Documentation. [Online]. Available: https://docs.python.org/3/library/xml.etree.elementtree.html (accessed Oct. 8, 2026).

[7] B. Staniar, "Local file save with Tableau Desktop Public Edition," Tableau Blog, Apr. 30, 2024. [Online]. Available: https://www.tableau.com/en-gb/blog/local-file-save-tableau-desktop-public-edition

[8] Tableau, "Save workbooks to Tableau Public," Tableau Desktop Help. [Online]. Available: https://help.tableau.com/current/pro/desktop/en-us/publish_workbooks_tableaupublic.htm (accessed Oct. 8, 2026).

[9] PCI Security Standards Council, "How can an entity meet PCI DSS requirements for PAN masking and truncation if it has migrated to 8-digit BINs?," FAQ 1492, Apr. 2024. [Online]. Available: https://www.pcisecuritystandards.org/faqs/1492/

[10] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

[11] National Center for Health Statistics, "Geographic division or region," Health, United States, Centers for Disease Control and Prevention. [Online]. Available: https://www.cdc.gov/nchs/hus/sources-definitions/geographic-region.htm (accessed Oct. 8, 2026).

[12] L. Sweeney, "k-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557-570, 2002, doi: 10.1142/S0218488502001648.

[13] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, Art. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[14] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, vol. 41, no. 10, pp. 4915-4928, 2014, doi: 10.1016/j.eswa.2014.02.026.

[15] Y.-A. Le Borgne, W. Siblini, B. Lebichot, and G. Bontempi, *Reproducible Machine Learning for Credit Card Fraud Detection: Practical Handbook*. Brussels, Belgium: Université Libre de Bruxelles, 2022. [Online]. Available: https://fraud-detection-handbook.github.io/fraud-detection-handbook/Chapter_2_Background/FDS.html

[16] K. Shenoy, "Credit Card Transactions Fraud Detection Dataset," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/kartik2112/fraud-detection

[17] B. Harris, "Sparkov_Data_Generation," GitHub repository. [Online]. Available: https://github.com/namebrandon/Sparkov_Data_Generation (accessed Oct. 8, 2026).

[18] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235-249, 2002, doi: 10.1214/ss/1042727940.

[19] A. Machanavajjhala, D. Kifer, J. Gehrke, and M. Venkitasubramaniam, "l-diversity: Privacy beyond k-anonymity," *ACM Transactions on Knowledge Discovery from Data*, vol. 1, no. 1, Art. 3, Mar. 2007, doi: 10.1145/1217299.1217302.

[20] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Art. 4(1) and Art. 4(5), Apr. 27, 2016. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[21] C. Dwork, "Differential privacy," in *Automata, Languages and Programming (ICALP 2006)*, Lecture Notes in Computer Science, vol. 4052. Berlin, Germany: Springer, 2006, pp. 1-12, doi: 10.1007/11787006_1.

Appendix A. Additional screenshots

These figures and listings support Chapters 3 and 4.

A.1 k-anonymity before and after generalization

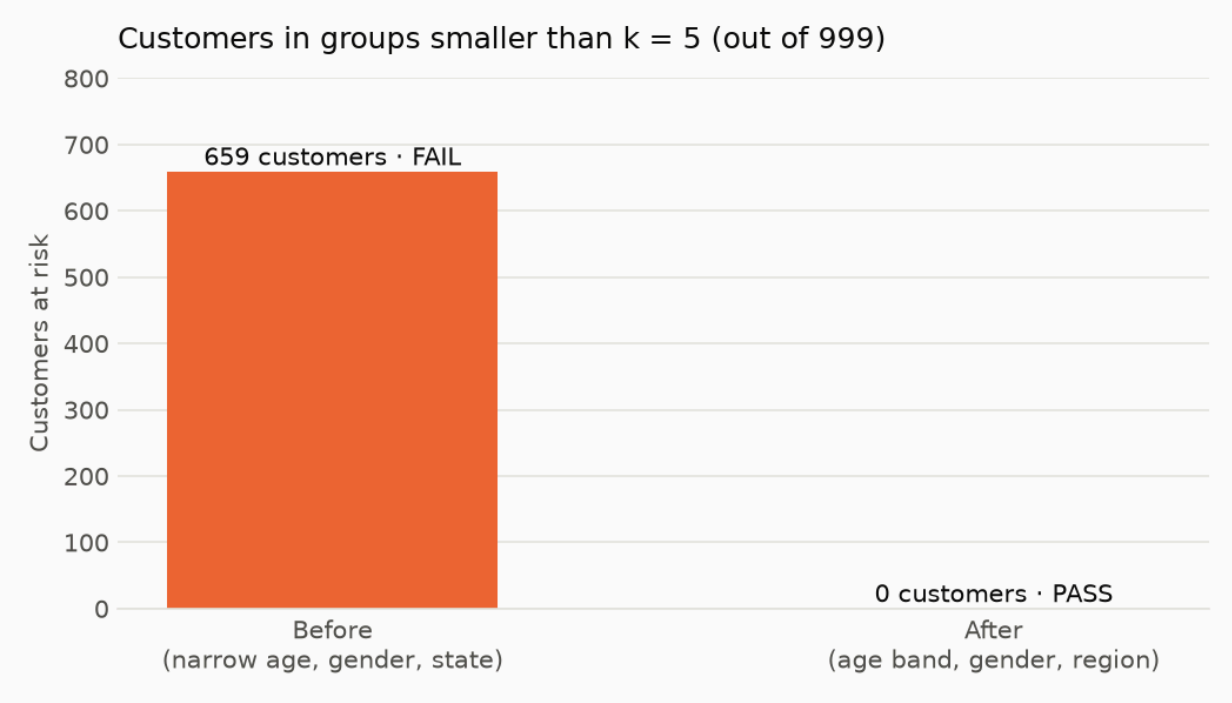


Figure A.1: Customers in groups smaller than k = 5 (out of 999). Before: narrow age band, gender and state (FAIL, 659 customers). After: age band, gender and region (PASS, 0 customers).

A.2 Threshold tuning on the training file

Table A.1 reproduces `outputs/rule_tuning.csv`.

Threshold	Flagged	TP	FP	FN	TN	Precision	Recall	F1	Accuracy
1	530,962	7,208	523,754	298	765,415	0.0136	0.9603	0.0268	0.5958
2	108,984	5,411	103,573	2,095	1,185,596	0.0496	0.7209	0.0929	0.9185
3	**9,357**	**3,934**	**5,423**	**3,572**	**1,283,746**	**0.4204**	**0.5241**	**0.4666**	**0.9931**
4	1,485	1,237	248	6,269	1,288,921	0.8330	0.1648	0.2752	0.9950

Table A.1: F1 score for each risk score threshold on the training file. Threshold 3 (bold) has the best F1 and was selected.

A.3 Project folder structure

```
CBCA-293-Security/
├── pyproject.toml, uv.lock     uv project (Python 3.13)
├── .python-version             3.13
├── data/
│   ├── raw/                    fraudTrain.csv, fraudTest.csv (Kaggle)
│   └── processed/              transactions_protected.csv and .hyper (Tableau input)
├── src/
│   ├── config.py               paths, salt, rule settings
│   ├── load_data.py            step 1: load, combine, clean, profile
│   ├── privacy.py              step 2: mask, hash, generalize, k-anonymity
│   ├── fraud_rules.py          steps 3-4: rules, score, metrics
│   ├── make_charts.py          step 5: report charts
│   ├── build_dashboard.py      step 6: writes the Tableau .twb and .twbx
│   ├── make_extract.py         writes the Tableau extract (.hyper)
│   └── run_pipeline.py         runs steps 1-6 in order
├── outputs/                    JSON/CSV results, charts/*.png
├── dashboard/                  fraud_dashboard.twb, fraud_dashboard.twbx, screenshots/
├── reports/                    00_front_matter.md ... 05_chapter5.md, appendix_a.md,
│                               presentation.md, assets/
└── docs/                       requirements, project understanding, decisions, plan
```

Figure A.2: Project folder structure

A.4 Output files

File	Size	Produced by	Content
`data/processed/transactions_protected.csv`	about 265 MB	`run_pipeline.py`	1,852,394 protected rows, 19 columns
`outputs/data_profile.json`	703 bytes	`load_data.py`	Dataset facts
`outputs/privacy_report.json`	1,224 bytes	`privacy.py`	k-anonymity before and after, example record, removed columns
`outputs/rule_tuning.csv`	294 bytes	`fraud_rules.py`	Train results for thresholds 1 to 4
`outputs/metrics.json`	1,562 bytes	`fraud_rules.py`	Learned settings and test metrics
`outputs/charts/fraud_rate_by_hour.png`	38,360 bytes	`make_charts.py`	Figure 3.2
`outputs/charts/fraud_rate_by_category.png`	84,752 bytes	`make_charts.py`	Figure 3.3
`outputs/charts/rule_comparison.png`	43,885 bytes	`make_charts.py`	Figures 3.4 and 4.2
`outputs/charts/confusion_matrix.png`	37,315 bytes	`make_charts.py`	Figures 3.5 and 4.3

File	Size	Produced by	Content
`outputs/charts/k_anonymity_before_after.png`	46,364 bytes	`make_charts.py`	Figures 3.6, 4.1 and A.1
`dashboard/fraud_dashboard.twb`	100,489 bytes	`build_dashboard.py`	Tableau workbook (12 worksheets, 6 calculated fields, 6 filters)
`data/processed/transactions_protected.hyper`	about 36 MB	`make_extract.py`	Tableau extract, 1,852,394 rows
`dashboard/fraud_dashboard.twbx`	about 21 MB	`build_dashboard.py`	Packaged workbook (dashboard + extract)

Table A.2: Files from one pipeline run

A.5 Additional Tableau screenshots

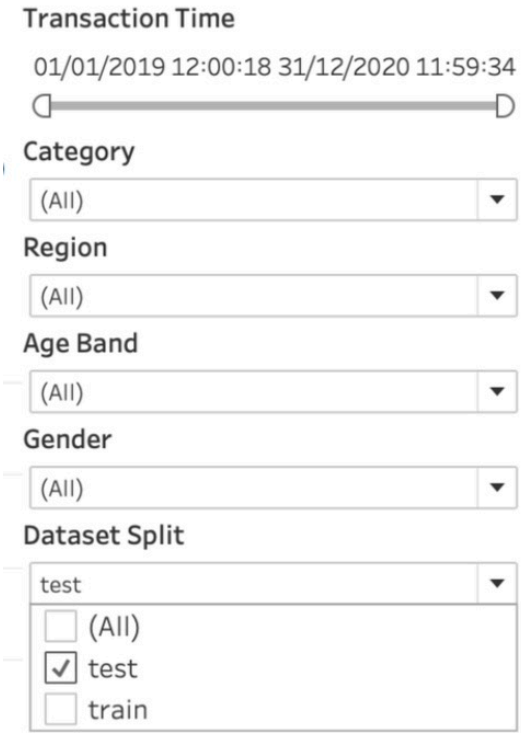


Figure A.3: The filter panel (date range, category, region, age band, gender, dataset split)

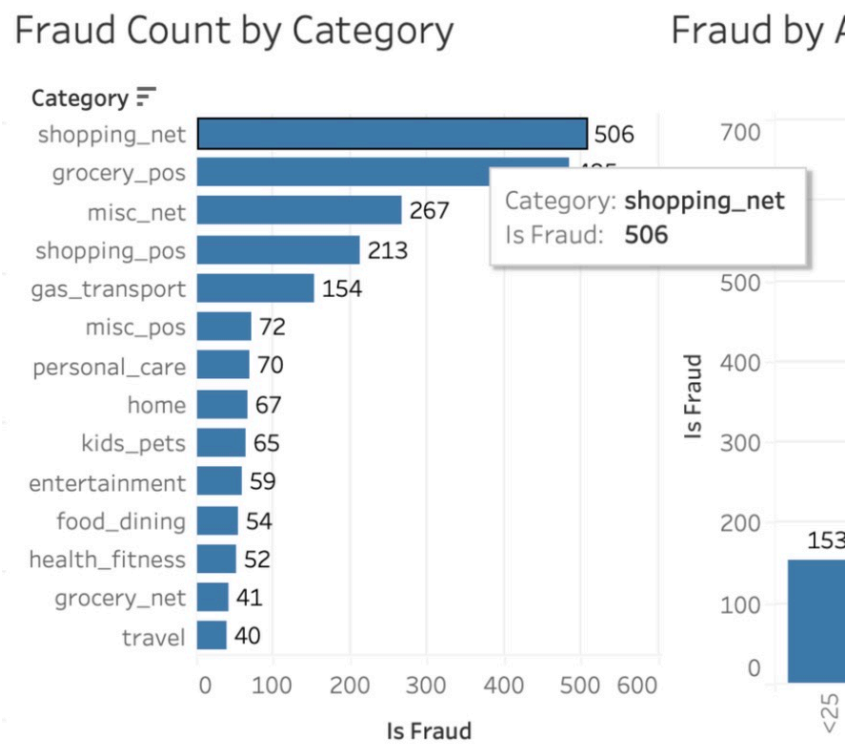


Figure A.4: A tooltip on the Fraud Count by Category chart, showing protected fields only